



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Detección y análisis de
ataques en sistemas
embebidos**



Presentado por Sara Abejón Pérez
en Universidad de Burgos — 8 de junio
de 2026

Tutor: Jaime Andrés Rincón Arango y Daniel
Urda Muñoz



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Jaime Andrés Rincón Arango y D. Daniel Urda Muñoz, profesores del departamento de Digitalización, área de Ciencia de la Computación e Inteligencia Artificial

Exponen:

Que el alumno D. Sara Abejón Pérez, con DNI 71829336Z, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Detección y análisis de ataques en sistemas embebidos.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 8 de junio de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. Jaime Andrés Rincón Arango

D. Daniel Urda Muñoz

Resumen

Este proyecto propone el desarrollo de un sistema para la detección on-edge de ataques de Denegación de Servicio (DoS) dirigidos a microcontroladores. El sistema permitirá aislar el proceso de detección en un microcontrolador de forma que, tanto el procesamiento como la clasificación de los flujos recibidos se realice sin intervención de otro dispositivo. La finalidad es demostrar la capacidad de los microcontroladores actuales para mantener su autonomía y seguridad frente a ataques volumétricos procesando los datos directamente en el dispositivo.

Descriptores

Ciberseguridad, Edge Computing, Internet de las Cosas (IoT), Detección de ataques, Denegación de Servicio (DoS), Aprendizaje automático, Random Forest, Microcontroladores, Procesamiento de tráfico de red.

Abstract

This project proposes the development of a system for the on-edge detection of Denial-of-Service (DoS) attacks targeting microcontrollers. The system will enable the detection process to be isolated within a microcontroller, such that both the processing and classification of incoming traffic are carried out without the intervention of any external device. The objective is to demonstrate the capability of modern microcontrollers to maintain autonomy and security against volumetric attacks by processing data directly on the device.

Keywords

Cybersecurity, Edge Computing, Internet of Things (IoT), Attack Detection, Denial-of-Service (DoS), Machine Learning, Random Forest, Microcontrollers, Network Traffic Processing.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
1.1. Contexto	1
1.2. Motivación	2
1.3. Solución	2
1.4. Repositorio	3
2. Objetivos del proyecto	5
2.1. Objetivos Generales	5
2.2. Objetivos Técnicos	5
2.3. Objetivos Personales	6
3. Conceptos teóricos	7
3.1. Sistemas embebidos y dispositivos IoT	7
3.2. Ciberataques	11
3.3. Aprendizaje Automático (Machine Learning)	19
4. Técnicas y herramientas	23
4.1. Técnicas utilizadas	23
4.2. Herramientas utilizadas	23
5. Aspectos relevantes del desarrollo del proyecto	27
5.1. Fase de experimentación	27

5.2. Fase de desarrollo	28
6. Trabajos relacionados	37
7. Conclusiones y Líneas de trabajo futuras	41
7.1. Conclusiones	41
7.2. Líneas de ttrabajo futuras	42
Bibliografía	45

Índice de figuras

3.1. Arduino Portenta H7	9
3.2. M5Stack Core2 ESP32	9
3.3. M5Stack LLM630 Compute Kit (AX630C)	10
3.4. Cyber Kill Chain	12
3.5. Ataque DoS	15
3.6. Ataque DDoS	16
3.7. SYN Flood	18
3.8. ICMP Flood	19
3.9. Algoritmo Random Forest	21
5.1. Matriz de confusión de la evaluación interna para totalTONIoT	30
5.2. Matriz de confusión de la evaluación interna para totalPropio	31
5.3. Matriz de confusión de la evaluación interna para mezclaTONIoT	32
5.4. Matriz de confusión de la evaluación interna para mezclaPropio	33

Índice de tablas

5.1. Resultados validación cruzada para totalTONIoT	29
5.2. Resultados validación cruzada para totalPropio	30
5.3. Resultados validación cruzada para mezclaTONIoT	31
5.4. Resultados validación cruzada para mezclaPropio	32
5.5. Resultados evaluación para totalTONIoT	34
5.6. Resultados evaluación para totalPropio	34
5.7. Resultados evaluación para mezclaTONIoT	35
5.8. Resultados evaluación para mezclaPropio	35

1. Introducción

1.1. Contexto

En la actualidad, el despliegue masivo del Internet de las Cosas (IoT) ha transformado la arquitectura de las redes modernas, integrando microcontroladores y sensores en una amplia gama de sectores, desde infraestructuras industriales hasta entornos domésticos. No obstante, este incremento en la cantidad de dispositivos interconectados ha conllevado una expansión proporcional de la superficie de exposición ante amenazas cibernéticas. Entre los diversos vectores de ataque, la Denegación de Servicio (DoS) destaca por su capacidad de comprometer la disponibilidad de servicios críticos mediante la saturación de recursos.

La detección de este tipo de intrusiones en dispositivos de bajos recursos presenta desafíos significativos en términos de eficiencia y autonomía. Tradicionalmente, la seguridad de red ha dependido de sistemas centralizados de análisis de tráfico; sin embargo, el paradigma actual se orienta hacia el Edge Computing o computación en el borde. Este enfoque busca desplazar la toma de decisiones al extremo de la red, permitiendo que el propio dispositivo realice el procesamiento de datos sin intervención de infraestructuras externas.

La rápida evolución de la inteligencia artificial y el aprendizaje automático ofrece nuevas oportunidades para mejorar la efectividad de estos sistemas de defensa. En este marco, el uso de técnicas de clasificación basadas en flujos de red permite identificar patrones de tráfico malicioso directamente en el dispositivo.

Este proyecto se centra en el desarrollo de un sistema de detección on-edge de ataques DoS dirigido a microcontroladores de alto rendimiento. El

objetivo es dotar a estos equipos de la capacidad para procesar, analizar y clasificar el tráfico de red de forma autónoma y en tiempo real, garantizando la seguridad y resiliencia de los sistemas embebidos frente a amenazas volumétricas en el punto de origen de los datos.

1.2. Motivación

La realización de este proyecto surge a partir de mi colaboración con el Grupo de Inteligencia Computacional Aplicada (GICAP) de la Universidad de Burgos en el proyecto Artificial Intelligence for Securing IoT Devices financiado por INCIBE. Durante parte del tercer y cuarto curso del grado, he tenido la oportunidad de investigar las crecientes capacidades de los microcontroladores y el potencial del Edge Computing para ejecutar modelos de inteligencia artificial de forma autónoma.

A lo largo de esta colaboración, se me propusieron diversas líneas de trabajo que sirvieron como guía para consolidar los conceptos teóricos en una implementación práctica. Como demostración del trabajo realizado, se planteó demostrar la viabilidad de trasladar la inteligencia al extremo de la red, aislando el proceso de detección para que el dispositivo sea capaz de procesar y clasificar flujos de red en tiempo real sin dependencia de infraestructuras externas.

La oportunidad de participar en un proyecto tan ambicioso ha sido una gran motivación para el desarrollo de este Trabajo Fin de Grado. Confío en que esta solución sirva como una prueba de concepto sólida sobre la viabilidad de descentralizar la ciberseguridad, un aspecto crítico en el ecosistema actual del Internet de las Cosas (IoT).

1.3. Solución

La solución propuesta consiste en el desarrollo de un sistema integral de detección de intrusiones que combina una infraestructura de ejecución on-edge con una plataforma de gestión centralizada. Para lograr esto, se crearán modelos de clasificación binaria basados en el algoritmo Random Forest, entrenados con flujos de tráfico de red que incluyen tráfico legítimo y variantes de ataque DoS (SYNFlood e ICMPFlood).

Los modelos de inteligencia artificial se integrarán en una plataforma M5Stack LLM630 Compute Kit basada en el SoC AX630C. Utilizando NFStream para la captura y análisis de tráfico en tiempo real, el siste-

ma será capaz de identificar, caracterizar y clasificar automáticamente los flujos de red dirigidos al dispositivo, facilitando la detección temprana de comportamientos anómalos o potencialmente maliciosos.

Además, se desarrollará una aplicación orientada a la monitorización de este entorno experimental, de forma que se permita la carga de modelos en microcontroladores, facilitando el testeo del rendimiento en tiempo real mediante el análisis de los logs generados por el dispositivo durante el procesamiento de red.

Para garantizar la eficiencia del sistema, la aplicación incorpora un módulo de usuarios y una política de retención de datos que automatiza el borrado de modelos inactivos tras un periodo de 40 días, asegurando un entorno de pruebas optimizado y escalable.

1.4. Repositorio

El código fuente y los recursos asociados al proyecto se encuentran disponibles en el repositorio de GitHub en la siguiente dirección:

<https://github.com/saraabejonperez/Detection-and-analysis-of-attacks-on-embedded-systems.git>

La estructura del repositorio está organizada de manera que facilite la comprensión y reproducción del trabajo realizado, incluyendo subdirectorios para los diferentes componentes del sistema, conjuntos de datos utilizados y documentos relacionados.

Para una explicación detallada del repositorio, consultar el anexo del manual del programador (**sección D.2** de los anexos).

2. Objetivos del proyecto

Este apartado explica de forma precisa y concisa cuales son los objetivos que se persiguen con la realización del proyecto. Se puede distinguir entre los objetivos marcados por los requisitos del software a construir, los objetivos de carácter técnico que plantea a la hora de llevar a la práctica el proyecto y los objetivos personales a conseguir por el alumno.

2.1. Objetivos Generales

- Desarrollar un sistema de detección on-edge capaz de clasificar en tiempo real flujos de red benignos y ataques de Denegación de Servicio (DoS) de forma autónoma, directamente en un microcontrolador.
- Crear una plataforma web integral para la gestión centralizada de los dispositivos, el despliegue remoto de los modelos de clasificación, y la monitorización y evaluación exhaustiva del sistema de detección en tiempo real.

2.2. Objetivos Técnicos

- Entrenar y evaluar un modelo de clasificación binaria basado en el algoritmo Random Forest, utilizando conjuntos de datos (archivos CSV) que contengan flujos benignos y ataques DoS específicos (SYNFlood e ICMPFlood), guardando la lista exacta de características empleadas para asegurar su replicabilidad.

- Integrar el modelo en un M5Stack LLM630 Compute Kit (AX630C), desarrollando un código que procese exclusivamente los flujos dirigidos a la IP del microcontrolador mediante la herramienta NFStream, clasificándolos en tiempo real y mostrando el resultado por pantalla.
- Desarrollar un panel de control web que permita a los usuarios registrar microcontroladores físicos, subir modelos de inteligencia artificial y asignarlos remotamente a los dispositivos correspondientes para iniciar la detección en tiempo real.
- Implementar un sistema de monitorización y alertas en la plataforma web que reciba avisos instantáneos desde el dispositivo físico ante la detección de un ataque, y que, al finalizar la prueba, recopile logs para generar estadísticas y gráficas detalladas sobre el tráfico analizado.
- Crear un entorno de evaluación de modelos en la web que permita testear la precisión de los clasificadores subidos utilizando datos de prueba locales, generando métricas de rendimiento, gráficas y matrices de confusión para validar su fiabilidad.
- Diseñar un sistema de gestión de usuarios y optimización de almacenamiento, permitiendo el acceso como usuario registrado (con historial persistente de dispositivos y modelos) o como invitado (sin retención de datos). Además, se deberá implementar un proceso automatizado que elimine los modelos tras 40 días ininterrumpidos sin ser utilizados en la web o en un dispositivo físico.

2.3. Objetivos Personales

- Aplicar los conocimientos adquiridos durante el Grado en Ingeniería Informática en un proyecto real y práctico.
- Profundizar en la aplicación práctica de modelos de aprendizaje automático y en los retos que supone su integración en dispositivos con recursos limitados bajo el paradigma del Edge Computing.
- Adquirir experiencia avanzada en el desarrollo web y arquitecturas distribuidas, logrando integrar con éxito la comunicación en tiempo real entre una aplicación web en la nube y el hardware físico.

3. Conceptos teóricos

3.1. Sistemas embebidos y dispositivos IoT

Sistemas Embebidos

Los **sistemas embebidos**[14] son dispositivos computacionales diseñados para realizar una función o un conjunto reducido de funciones en específico dentro de un sistema mayor, que operan bajo restricciones estrictas de consumo energético, memoria, capacidad de procesamiento y coste. A diferencia de los sistemas de propósito general, los sistemas embebidos se diseñan con una finalidad concreta, integrándose estrechamente con el hardware.

Desde una perspectiva arquitectónica, un sistema embebido se estructura fundamentalmente en torno a un **microcontrolador (MCU)** o **microprocesador (MPU)**, el cual se apoya en una combinación de memoria volátil y no volátil (RAM, Flash, eMMC, etc.) para el almacenamiento y ejecución de datos. Este núcleo se complementa con diversas interfaces de comunicación, que incluyen desde GPIO, UART y CAN hasta conectividad avanzada como Ethernet, Wi-Fi o Bluetooth; y un software específico, ya sea en forma de firmware, sistemas operativos de tiempo real (RTOS) o distribuciones de Linux embebido, que coordina el funcionamiento de todo el conjunto.

La evolución de la microelectrónica ha permitido integrar mayores capacidades de procesamiento en dispositivos de bajo consumo, favoreciendo la convergencia entre sistemas embebidos tradicionales y plataformas capaces de ejecutar algoritmos de inteligencia artificial en el borde (**edge AI**) [?]. Los sistemas embebidos, al integrarse en objetos o sistemas más complejos, desempeñan un papel fundamental en el Internet de las cosas (IoT).

Internet of Things

El concepto de **Internet de las Cosas (IoT)** se refiere a la interconexión de objetos físicos dotados de capacidades de sensorización, procesamiento y comunicación a través de redes IP. Según Atzori et al. (2010)[6], el IoT puede definirse como una infraestructura que conecta objetos identificables de forma única, permitiendo la integración entre el mundo físico y el digital.

En el contexto actual, los dispositivos Internet of Things (IoT) [?] se definen por la integración sinérgica de la computación en el borde para el procesamiento local de datos, respaldada por una robusta conectividad inalámbrica o cableada. Esta estructura se apoya en el uso de protocolos ligeros, diseñados para optimizar el consumo de recursos, lo que facilita una comunicación eficiente y una integración fluida con servicios en la nube para el análisis y almacenamiento a gran escala.

El despliegue masivo de estos dispositivos en entornos industriales, urbanos y domésticos ha incrementado significativamente la superficie de ataque, lo que plantea importantes desafíos en materia de ciberseguridad.

En el marco de este trabajo, se emplean distintos dispositivos representativos de diferentes categorías dentro del ecosistema embebido, que permiten evaluar soluciones de detección de ataques en entornos con recursos limitados.

Arduino Portenta H7

La **Arduino Portenta H7**[4] es una placa de desarrollo orientada a aplicaciones industriales, IoT avanzado y edge computing. Está basada en el microcontrolador STM32H747 de STMicroelectronics, que integra una arquitectura dual-core ARM Cortex-M7 + Cortex-M4. Las características técnicas más relevantes de este dispositivo son:

- Arquitectura dual-core de hasta 480 MHz (Cortex-M7).
- Soporte para ejecución de modelos de aprendizaje automático mediante TensorFlow Lite Micro.
- Conectividad Wi-Fi y Bluetooth.
- Interfaces industriales (UART, SPI, I2C, CAN).
- Soporte para RTOS o ejecución bare-metal.

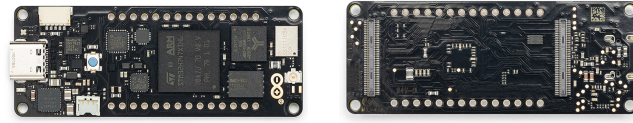


Figura 3.1: Arduino Portenta H7

Su arquitectura híbrida permite delegar tareas críticas de tiempo real al núcleo Cortex-M4, mientras el Cortex-M7 ejecuta tareas de mayor carga computacional, como procesamiento de señales o inferencia de modelos de clasificación. Esta capacidad la convierte en una plataforma adecuada para implementar mecanismos de detección en el borde con restricciones de latencia.

M5Stack Core2 ESP32

El **M5Stack Core2 ESP32**^[21] es un kit de desarrollo basado en el microcontrolador ESP32, ampliamente utilizado en aplicaciones IoT debido a su bajo coste y conectividad integrada. Entre las características principales de este dispositivo destaca su microcontrolador dual-core Xtensa LX6, que ofrece un equilibrio excepcional entre potencia y eficiencia. Este hardware se potencia con conectividad nativa Wi-Fi y Bluetooth, una pantalla táctil integrada para una interacción directa y un diseño orientado al bajo consumo energético, ideal para aplicaciones portátiles. Además, su versatilidad se ve respaldada por un amplio ecosistema de librerías y un sólido soporte comunitario que simplifica significativamente el proceso de desarrollo.



Figura 3.2: M5Stack Core2 ESP32

El ESP32 se ha convertido en una de las plataformas más extendidas en proyectos IoT académicos e industriales. Sin embargo, su capacidad de procesamiento y memoria es limitada en comparación con sistemas basados en

Linux, lo que lo convierte en un escenario adecuado para evaluar la viabilidad de modelos de detección ligeros optimizados para microcontroladores.

Desde el punto de vista de la seguridad, su uso masivo y exposición frecuente a redes inalámbricas lo convierten en un objetivo atractivo para ataques de red, incluyendo escaneos automatizados y ataques de denegación de servicio.

M5Stack LLM630 Compute Kit (AX630C)

El M5Stack LLM630 Compute Kit (AX630C) [22] se sitúa en una categoría superior dentro de las plataformas de computación embebida. Su arquitectura basada en un SoC (System-on-Chip) con procesadores ARM Cortex-A le proporciona la capacidad necesaria para ejecutar sistemas operativos completos, como Ubuntu 22.04 LTS, y aplicaciones avanzadas de procesamiento en el borde (Edge Computing). A diferencia de los microcontroladores tradicionales, esta plataforma ofrece una mayor capacidad de memoria RAM y almacenamiento, así como interfaces de comunicación de altas prestaciones, entre las que destacan Ethernet y Wi-Fi. Además, incorpora soporte específico para la aceleración de inferencia mediante modelos de inteligencia artificial, lo que la convierte en una solución idónea para aplicaciones de visión artificial, aprendizaje automático y análisis inteligente de datos en tiempo real.

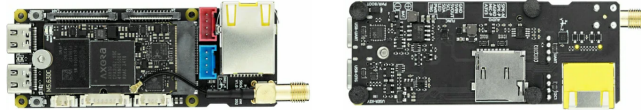


Figura 3.3: M5Stack LLM630 Compute Kit (AX630C)

Este tipo de plataforma pertenece a la categoría de sistemas de computación embebida de alto rendimiento para aplicaciones de Edge AI. Gracias a su arquitectura basada en Linux y a sus capacidades de aceleración para inteligencia artificial, permite la ejecución de modelos de aprendizaje automático complejos, incluyendo redes neuronales desarrolladas con TensorFlow Lite, PyTorch o frameworks equivalentes, así como la implementación de servicios de monitorización, análisis de datos y visualización en tiempo real.

Desde el punto de vista de ciberseguridad, al ejecutar un sistema operativo completo, su superficie de ataque es mayor que la de un microcontrolador bare-metal, ya que incorpora servicios de red, la pila TCP/IP completa, un sistema de archivos y posibles procesos concurrentes.

Por tanto, constituye un entorno idóneo para estudiar ataques de red reales y evaluar el impacto de estos en dispositivos de recursos intermedios.

3.2. Ciberataques

Los **ciberataques** son acciones intencionadas orientadas a comprometer la seguridad de un sistema, red o dispositivo, explotando las vulnerabilidades técnicas, los errores de configuración o las debilidades de diseño que estos presenten. La finalidad de los ataques cibernéticos no es única, ya que depende del objetivo personal perseguido por el autor del ataque; sin embargo, todos los ataques atentan contra los pilares fundamentales de la seguridad informática: la **triada CIA**.

La definición de ciberataque en el ámbito de los sistemas embebidos e IoT adquiere una dimensión adicional por su interacción con el entorno físico. Un ataque a este tipo de sistemas puede provocar, además de los daños comunes a sistemas digitales, consecuencias tangibles como la interrupción de procesos industriales, la manipulación de sensores o la degradación de sistemas de control.

Desde el punto de vista técnico, un ciberataque tiene un ciclo de vida (Figura 3.4) estructurado en varias fases que pueden identificarse en la cadena **Cyber Kill Chain**[16], un modelo que describe cada una de las etapas de un ataque con el objetivo de detectar, detener y recuperarse en caso de sufrir uno. Las etapas del Cyber Kill Chain son:

1. Reconocimiento: el ciberdelincuente recopila información sobre el objetivo del ataque, valorando los métodos a usar y su probabilidad de éxito.
2. Preparación: el ataque se prepara de forma específica sobre el objetivo fijado.
3. Distribución: en esta fase se produce la transmisión del ataque.
4. Explotación: en esta fase se produce la ejecución del ataque.
5. Instalación: el atacante instala el malware en la víctima. Esta etapa no es universal, ya que hay tipos de ataques que no necesitan instalación.
6. Comando y control: el atacante ya tiene el control del sistema víctima y es capaz de ejecutar acciones maliciosas desde un servidor central C&C (Command and Control).

7. Acciones sobre los objetivos: el atacante se hace con los datos e intenta expandir el ataque a otras víctimas. Esta expansión hace que el ciclo de vida de un ciberataque sea cíclico, ya que las etapas se volverían a ejecutar en las nuevas víctimas infectadas en esta fase.

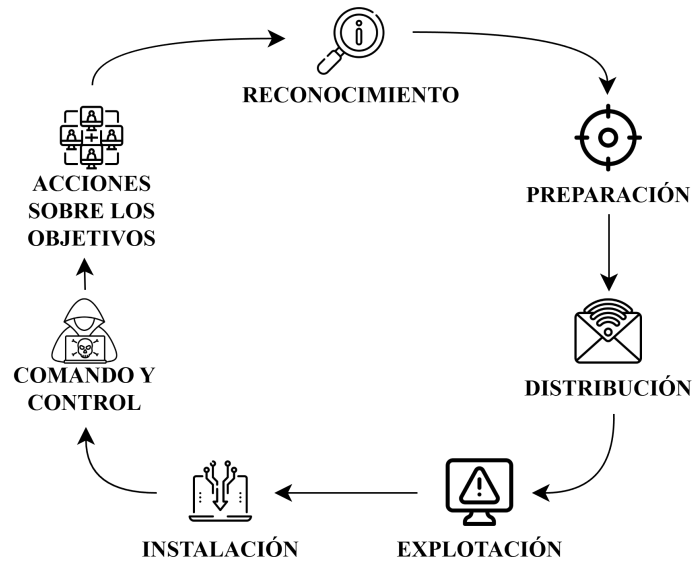


Figura 3.4: Cyber Kill Chain

Debido a su diversa finalidad, los ciberataques se pueden clasificar atendiendo a distintos criterios, como su objetivo, la técnica empleada, la capa del modelo OSI afectada o el impacto generado. Los principales tipos de ataques más relevantes en el ámbito de redes y sistemas embebidos son[12]:

■ Ataques pasivos

Reciben este nombre porque no alteran directamente el funcionamiento del sistema[9]. Su objetivo principal es obtener información de forma indetectable.

Sniffing de red: Es un tipo de ataque pasivo en el que se capturan paquetes en tránsito mediante herramientas de análisis de tráfico para obtener información confidencial, ya que en redes no cifradas, permite acceder a credenciales o datos sensibles.

Análisis de tráfico: Es un tipo de ataque pasivo que se basa en la observación de patrones de comunicación (frecuencia, tamaño

de paquetes, destinos) para inferir comportamientos del sistema, incluso cuando el contenido está cifrado.

La peligrosidad de este tipo de ataques radica en la dificultad de su detección, ya que no implican alterar datos o recursos del sistema; y en que pueden servir como fase previa a ataques más agresivos.

■ Ataques activos

Son ataques que pretenden alterar, manipular, destruir o interrumpir el funcionamiento normal de un sistema o red[13].

Man-in-the-Middle(MitM): Es un tipo de ataque activo en el que el atacante intercepta la comunicación entre dos entidades sin que estas lo detecten, de forma que puede alterar el mensaje antes de que lo reciba el destinatario original.

IP Spoofing: Es un tipo de ataque activo donde el atacante suplanta la dirección IP de origen para ocultar su identidad real o para simular tráfico legítimo.

Replay attack: Es un tipo de ataque activo basado en el reenvío de mensajes previamente capturados para engañar al sistema receptor.

■ Ataques basados en explotación de vulnerabilidades

También llamados *exploits*[17]. Esta categoría recoge todos aquellos programas, fragmentos de código o técnicas que aprovechan una vulnerabilidad en un sistema, aplicación o dispositivo para realizar acciones no autorizadas.

Buffer overflow: Es un exploit que fuerza a una aplicación a sobrescribir memoria, escribiendo datos fuera de los límites de memoria asignados, para redirigir la ejecución hacia código malicioso. Afecta principalmente a software escrito en lenguajes con gestión manual de memoria, como C o C++.

Inyección de código: Es un tipo de ataque que aprovecha entradas no validadas para introducir instrucciones maliciosas en el sistema.

Explotación de firmware vulnerable: Es un tipo de exploit que se beneficia del uso de versiones obsoletas sin parches de seguridad para acceder a los sistemas. Este tipo de ataques afectan mucho a los dispositivos embebidos por su falta de actualizaciones automáticas.

■ Malware y botnets

Es todo programa o aplicación informática diseñado para robar información o tomar el control del sistema del equipo en el que se ejecuta[23]. Este tipo de software se instala en el equipo víctima sin el conocimiento de su propietario y realiza funciones con fines dañinos sin que este se percate.

Virus: Malware con el objetivo de alterar el funcionamiento del dispositivo infectado, que necesita de interacción para su propagación.

Gusano: Malware capaz de replicarse y trasladarse de un dispositivo infectado a otros a través de la red.

Troyano: Malware camuflado, es decir, se presenta como software legítimo y ejecuta acciones no deseadas en segundo plano.

Botnets IoT: Redes de dispositivos IoT comprometidos que son controlados por un ciberdelincuente para realizar ataques en conjunto[10]. Los dispositivos se añaden a este tipo de redes por infección de malware.

Además de estos, un ciberataque muy común es el de Denegación de Servicio. Este tipo de ataque es el tratado en este estudio.

Ataques de Denegación de Servicio

Un ataque de denegación de servicio[15] (Denial of Service, DoS) es un tipo de ciberataque en el que un actor malicioso busca imposibilitar que una máquina u otro dispositivo esté disponible para los usuarios a los que va dirigido, interrumpiendo el funcionamiento normal del mismo. El objetivo principal de este tipo de ataques es degradar o interrumpir la disponibilidad de un servicio, agotando los recursos del sistema objetivo o explotando vulnerabilidades en los protocolos de comunicación. Esto se consigue al sobrecargar o inundar la máquina objetivo con solicitudes hasta que el tráfico normal es incapaz de ser procesado, lo que provoca una denegación de servicio a los usuarios.

Los ataques DoS constituyen una de las amenazas más relevantes en entornos de red, especialmente en sistemas embebidos e IoT, caracterizados por recursos limitados de procesamiento, memoria y energía. En el contexto de sistemas embebidos como ESP32, Raspberry Pi o plataformas basadas en ARM Cortex, la superficie de ataque aumenta debido a las limitaciones en mecanismos de defensa avanzados, el uso frecuente de pilas TCP/IP ligeras,

las configuraciones por defecto inseguras y la exposición directa a redes inalámbricas (Wi-Fi) o entornos Ethernet no segmentados.

Por todo esto, su limitada capacidad para gestionar grandes volúmenes de tráfico y su frecuente despliegue en redes poco protegidas, los dispositivos IoT son particularmente vulnerables a ataques de denegación de servicio[20].

Los ataques DoS pueden clasificarse según dos criterios principalmente: según el **número de fuentes de ataque** y según la **capa del modelo OSI afectada** por el ataque.

Atendiendo a la fuente atacante, existen dos técnicas de este tipo de ataques, diferenciadas por la cantidad de ordenadores o IP's que provocan la denegación de servicio a la máquina objetivo.

DoS (Denial of Service): En este tipo de ataques se genera una cantidad masiva de peticiones al servicio desde una misma máquina o dirección IP, consumiendo así los recursos que ofrece el servicio hasta que llega un momento en que no tiene capacidad de respuesta y comienza a rechazar peticiones, materializando la denegación del servicio.

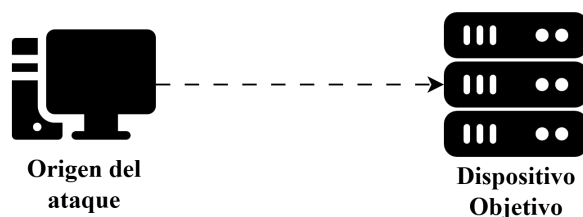


Figura 3.5: Ataque DoS

DDoS (Distributed Denial of Service): En este tipo de ataques se realizan peticiones o conexiones al mismo tiempo y hacia el mismo servicio objeto del ataque, empleando un gran número de ordenadores o direcciones IP.

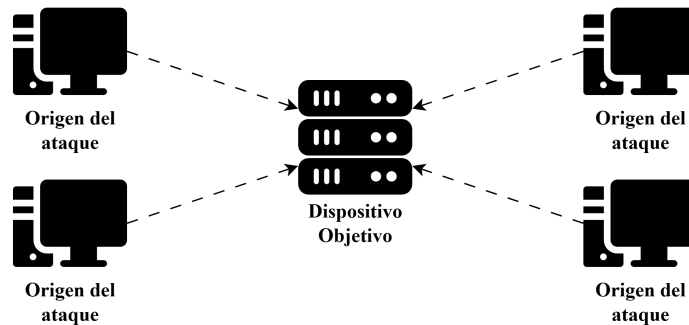


Figura 3.6: Ataque DDoS

Por otro lado, se pueden clasificar tres categorías de ataques DoS según el método o la capa del modelo OSI a la que afectan[18].

Ataques volumétricos: Crean la denegación de servicio al saturar el ancho de banda disponible del servidor de destino. Esto ocurre al enviar grandes volúmenes de datos, lo que provoca un aumento repentino del tráfico en el servidor. Un ejemplo de ataque volumétrico es ICMP Flood.

Ataques de protocolo: Explotan debilidades en la implementación de protocolos en las capas de red y transporte (capas 3 y 4 del modelo OSI), creando la denegación de servicio al saturar los recursos del servidor o de la red. Un ejemplo de ataque de protocolo es SYN Flood.

Ataques a la capa de aplicación: Agotan los recursos del servidor de destino utilizando sitios web DDoS. Se ejecutan numerosas solicitudes HTTP, saturando el servidor de destino, mientras el atacante responde cargando numerosos archivos y ejecutando las consultas de base de datos necesarias para crear una página web. Un ejemplo de ataque a la capa de aplicación es HTTP Flood.

De entre todas las categorías mencionadas, este proyecto centra su estudio en dos de las amenazas más críticas y recurrentes que afectan a los sistemas embebidos y dispositivos IoT : los ataques de protocolo y los ataques volumétricos. Debido a las restricciones inherentes de procesamiento y memoria de estos dispositivos , la explotación de protocolos de comunicación fundamentales a través de técnicas como SYN Flood o la saturación del ancho de banda mediante ICMP Flood resultan devastadoras. A continuación,

se detallan las características y el funcionamiento de estos dos vectores de ataque que conforman la base de la experimentación.

Ataques SYN Flood

El ataque **SYN Flood** es un ataque de denegación de servicio a nivel de protocolo que explota el mecanismo de establecimiento de conexión del protocolo TCP (Capa 4 del modelo OSI). En condiciones normales, una conexión TCP se inicia mediante un proceso conocido como saludo a tres vías (Three-way Handshake). El cliente envía un paquete SYN (sincronización), el servidor responde con un SYN-ACK (sincronización-reconocimiento) y el cliente finaliza el proceso enviando un paquete ACK (reconocimiento), estableciendo así la conexión.

En un ataque SYN Flood, el atacante envía una cantidad masiva de peticiones SYN al dispositivo objetivo de forma continua, a menudo utilizando direcciones IP de origen falsificadas (IP Spoofing). El servidor, al recibir estas peticiones, reserva recursos en memoria para cada conexión y responde con paquetes SYN-ACK, quedando a la espera del ACK final. Como las direcciones IP de origen son falsas o el atacante simplemente ignora la respuesta, el paquete ACK nunca llega. Esto genera una acumulación de conexiones semiabiertas (estado **En espera**) que terminan por agotar la tabla de conexiones del servidor. Una vez que se agotan estos recursos, el dispositivo es incapaz de aceptar nuevas conexiones legítimas, resultando en la denegación del servicio, tal y como se ilustra en la siguiente Figura 3.7.

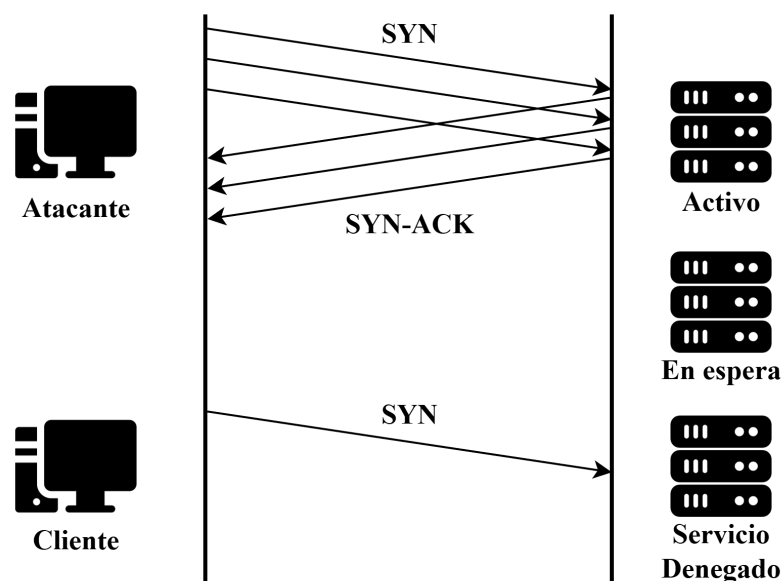


Figura 3.7: SYN Flood

Ataques ICMP Flood

El ataque ICMP Flood (Inundación ICMP o Ping Flood) es un ataque de denegación de servicio de tipo volumétrico. Su objetivo principal no es agotar la tabla de conexiones, sino consumir todo el ancho de banda disponible y los recursos de procesamiento del dispositivo objetivo o de su infraestructura de red. Este ataque abusa del Protocolo de Mensajes de Control de Internet (ICMP), diseñado originalmente para tareas de diagnóstico y control de errores en redes IP.

La técnica consiste en enviar un volumen abrumador de paquetes ICMP Echo Request (conocidos comúnmente como "pings") a la máquina víctima. El protocolo dicta que el dispositivo receptor debe procesar cada solicitud y formular una respuesta mediante un paquete ICMP Echo Reply. Si el atacante cuenta con un ancho de banda superior al de la víctima o utiliza una botnet para realizar un ataque distribuido (DDoS), el dispositivo embebido se verá rápidamente sobrepasado por la carga de procesamiento y la saturación de su enlace de red. La Figura 3.8 representa cómo el tráfico legítimo es bloqueado o ralentizado debido a esta inundación constante de peticiones.

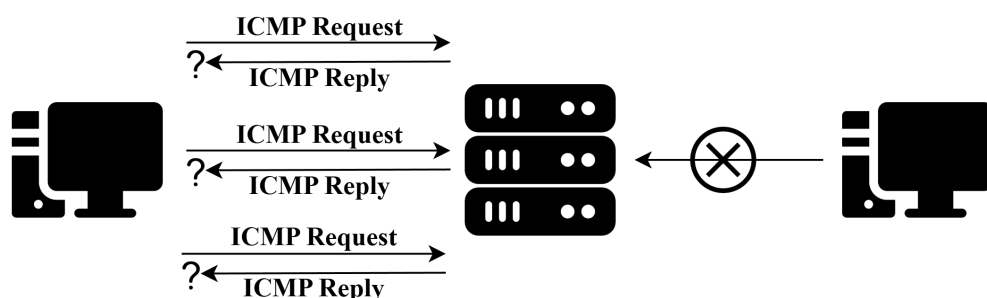


Figura 3.8: ICMP Flood

3.3. Aprendizaje Automático (Machine Learning)

El Aprendizaje Automático o Machine Learning[8] es una rama de la inteligencia artificial que permite a los sistemas aprender y mejorar a partir de la experiencia sin ser programados explícitamente para ello. El Machine Learning abarca diferentes métodos para solucionar distintos tipos de problemas. Las técnicas más destacadas son:

- **Clasificación**, para la asignación de una clase a las observaciones que poseen características de esas mismas clases.
- **Regresión**, para la predicción de valores continuos en lugar de etiquetas discretas.
- **Clustering**, para la agrupación de observaciones similares en subconjuntos llamados clústeres. A diferencia de las clases en la clasificación, los clústeres no están previamente definidos.

Además de los tipos de problemas que trata de solucionar, dentro del Aprendizaje Automático se distinguen tres tipos de aprendizaje dependiendo del modelo de entrenamiento usado:

1. **Aprendizaje supervisado:** Este tipo de aprendizaje utiliza conjuntos de datos previamente etiquetados para el entrenamiento de los algoritmos, lo que implica que cada instancia de entrada tiene asociada la salida correcta correspondiente. El objetivo de esta técnica es que el

algoritmo aprenda patrones y relaciones subyacentes entre las entradas y las salidas para predecir con precisión resultados en datos nuevos no vistos anteriormente.

2. **Aprendizaje no supervisado:** Este modelo de aprendizaje usa datos no estructurados (sin etiquetar) para aprender patrones. A diferencia del aprendizaje supervisado, la precisión de la salida no se conoce de antemano; el algoritmo aprende de los datos sin intervención humana y los categoriza en grupos según los atributos.
3. **Aprendizaje por refuerzo:** Esta técnica basa el aprendizaje de los algoritmos en una serie de experimentos de prueba y error, donde los agentes autónomos aprenden a realizar una tarea definida a través de un ciclo de retroalimentación hasta que su rendimiento esté dentro de un rango deseable. Para ello, el agente recibe un refuerzo positivo cuando realiza la tarea de forma correcta y un refuerzo negativo cuando tiene un rendimiento bajo.

En el contexto de la ciberseguridad y la detección de intrusiones, el Machine Learning permite analizar grandes volúmenes de tráfico de red para identificar patrones anómalos que caracterizan a un ciberataque.

El modelo desarrollado en este trabajo se enmarca en el aprendizaje supervisado mediante clasificación binaria. La clasificación binaria es un tipo concreto de clasificación que consiste en asignar datos de entrada en una de dos clases excluyentes.

En este enfoque, se proporcionan ejemplos de tráfico de red junto con el tipo de flujo que es (tráfico benigno o ataque DoS). A partir de estos datos, el modelo infiere las reglas subyacentes para poder predecir la etiqueta de nuevos flujos de red nunca antes vistos. El algoritmo seleccionado para realizar la clasificación del tráfico es el Random Forest.

Random Forest

El algoritmo **Random Forest**^[19] es un potente método de aprendizaje supervisado que pertenece a la familia de los modelos de aprendizaje conjunto (ensemble learning). Su funcionamiento se basa en la combinación de múltiples modelos predictivos más simples para generar un clasificador robusto, preciso y altamente resistente al ruido.

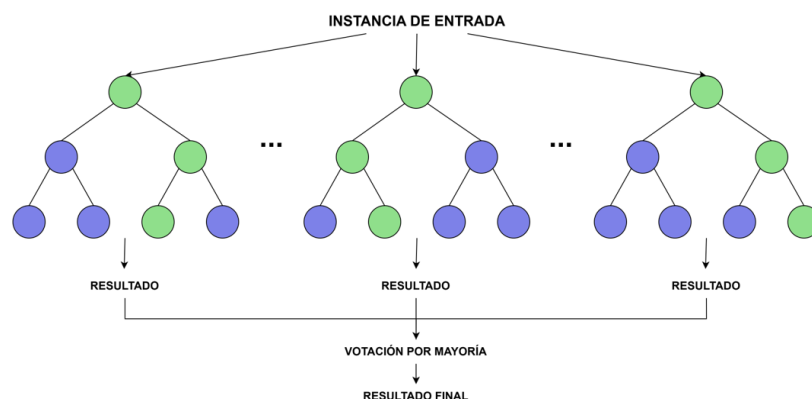


Figura 3.9: Algoritmo Random Forest

La estructura central de este algoritmo es el árbol de decisión. Se trata de un modelo en forma de diagrama de flujo donde cada nodo interno evalúa una característica específica de los datos, las ramas representan las reglas de decisión derivadas de esa evaluación, y los nodos hoja determinan la etiqueta de la clase final. Aunque son modelos muy intuitivos, los árboles de decisión individuales presentan una gran desventaja: son muy propensos al sobreajuste (overfitting) y al sesgo, volviéndose excesivamente sensibles a los cambios o ruidos presentes en el conjunto de entrenamiento.

Para mitigar las debilidades de los árboles individuales, Random Forest implementa una técnica conocida como Bagging (Bootstrap Aggregating). Durante la fase de entrenamiento, el algoritmo construye un *bosque* compuesto por múltiples árboles de decisión independientes. Cada uno de estos árboles se entrena utilizando un subconjunto aleatorio, con remplazo, de los datos originales. Además, en cada división de los nodos, el algoritmo solo evalúa un subconjunto aleatorio de las características disponibles. Esta doble aleatoriedad reduce drásticamente la correlación entre los árboles y evita que el modelo final memorice los datos de entrenamiento.

En la fase de clasificación, los datos de entrada recorren todos los árboles que componen el bosque, y cada árbol emite su propia predicción individual o "voto". El Random Forest utiliza como mecanismo de inferencia la votación por mayoría, de forma que los datos de entrada se clasifican en la clase que ha obtenido la mayoría absoluta de los votos.

En el contexto de este trabajo, la elección de Random Forest resulta especialmente adecuada. Al combinar el conocimiento de múltiples modelos "débiles", logra equilibrar el sesgo y la varianza, generando un modelo final mucho más fuerte. Además, su alta eficiencia computacional durante la

fase de inferencia lo convierte en un candidato ideal para ser ejecutado en tiempo real dentro de sistemas con recursos limitados, como es el caso de un microcontrolador.

4. Técnicas y herramientas

4.1. Técnicas utilizadas

Durante el desarrollo de este proyecto, se adoptaron metodologías ágiles para gestionar la evolución del trabajo y asegurar una integración exitosa del sistema.

SCRUM

La metodología **SCRUM** es un marco de trabajo ágil pensado para el desarrollo y mantenimiento de proyectos complejos. Esta metodología se estructura en ciclos de trabajo llamados **sprints** que, en el caso de este proyecto, tuvieron una duración de dos semanas. En cada sprint se entrega un producto funcional, lo que permite una evolución continua del proyecto.

Tras cada iteración, se valoraba el trabajo realizado y se procedía a la planificación de la siguiente iteración, generando las tareas que se realizarían en la próxima iteración. Esta metodología permitió una adaptación continua ante los posibles problemas técnicos y facilitó la división de tareas entre el entrenamiento del modelo, la programación del microcontrolador y la creación de la plataforma web.

Para un análisis más detallado del proceso de gestión del proyecto, debe consultar el **Apéndice A.2** de los anexos.

4.2. Herramientas utilizadas

Para facilitar la implementación técnica, la gestión del código y la documentación, se hizo uso de las siguientes herramientas de software:

Herramientas de desarrollo

- **Visual Studio Code:** Entorno de desarrollo integrado (IDE) empleado como editor y depurador principal de código fuente.
- **Python:** Lenguaje de programación principal utilizado en todo el proyecto, seleccionado por su robustez en el aprendizaje automático, así como por su flexibilidad para el desarrollo de la plataforma web.

Entre las bibliotecas utilizadas, destacar:

- **Scikit-Learn:** Librería fundamental de aprendizaje automático utilizada para el diseño, entrenamiento y evaluación del modelo de clasificación binaria basado en el algoritmo Random Forest. Proporcionó también las herramientas necesarias para obtener métricas de validación y matrices de confusión.
- **NFStream:** Herramienta de análisis de tráfico de red empleada en la fase experimental para el procesamiento de los paquetes en tiempo real en el microcontrolador M5Stack.
- **Pandas y NumPy:** Librerías de análisis y manipulación de datos utilizadas para la lectura, limpieza y transformación de los conjuntos de datos que contenían el tráfico.

Herramientas de despliegue

- **Docker Desktop:** Plataforma de contenerización utilizada para empaquetar la aplicación web y sus dependencias en contenedores aislados, garantizando que la plataforma funcione de manera idéntica y consistente en diferentes entornos de desarrollo y producción, facilitando un despliegue seguro y eficiente.
- **Flask:** Micro-framework de Python utilizado para el desarrollo de la aplicación web.

Herramientas de planificación y diseño

- **GitHub:** Plataforma de alojamiento de repositorios basada en Git usada para el control de versiones. Además, con **GitHub Projects** se generó un tablero para organizar las tareas en diferentes sprints e implementar el seguimiento de SCRUM.

- **Draw.io:** Herramienta gráfica en la nube utilizada para la creación de esquemas, diagramas de arquitectura del sistema y flujos de comunicación.
- **Overleaf y LaTeX:** Plataforma online y sistema de composición de textos utilizados para la redacción de la memoria del proyecto.

5. Aspectos relevantes del desarrollo del proyecto

En este capítulo se recogen los aspectos más importantes durante el desarrollo del proyecto. Se incluyen los detalles más relevantes de las fases de análisis, diseño, implementación y resultados.

5.1. Fase de experimentación

La primera fase del proyecto se centró en la investigación y experimentación práctica para definir el contexto tecnológico adecuado.

Inicialmente, se evaluaron dos enfoques distintos para la detección de ataques de Denegación de Servicio:

- la detección mediante reglas heurísticas basadas en umbrales estáticos de tráfico,
- y la detección mediante modelos de Inteligencia Artificial.

Paralelamente, se realizaron pruebas de viabilidad sobre diferentes arquitecturas de hardware embebido, concretamente el Arduino Portenta H7, el M5Stack Core2 ESP32 y el M5Stack LLM630 Compute Kit (AX630C).

Los resultados de estas pruebas, cuyos registros y códigos de prueba se encuentran documentados en el repositorio de GitHub del proyecto, evidenciaron que la detección heurística resultaba rígida y propensa a falsos positivos en entornos de red dinámicos. Por ello, se justificó el uso de modelos de aprendizaje automático, capaces de inferir patrones anómalos complejos.

En cuanto al hardware, el M5Stack LLM destacó por su mayor capacidad de procesamiento y compatibilidad con sistemas operativos basados en Linux, un requisito indispensable para poder ejecutar herramientas avanzadas de análisis de tráfico como NFStream y librerías de Machine Learning de forma nativa.

5.2. Fase de desarrollo

Una vez establecido el entorno tecnológico, se procedió al desarrollo integral del sistema, priorizando la creación de un modelo robusto y su posterior integración en tiempo real.

Creación de modelos

El proceso de entrenamiento se diseñó con un enfoque riguroso para asegurar que el modelo pudiera operar de forma fiable en el mundo real, donde la estructura del tráfico varía significativamente respecto a los entornos de laboratorio.

Dataset

Para el entrenamiento y la evaluación se utilizaron dos conjuntos de datos de distinta naturaleza:

- El dataset público **TONIoT**, un estándar en la investigación de ciberseguridad [1].
- Un dataset de flujos generado en un entorno controlado por el grupo GICAP: **CyberFlowIoT-GICAP**: Labelled Flow-Based Network Traffic Dataset for Cyberattack Detection [3].

Una decisión de diseño crítica fue la creación de un entorno de evaluación unificado para medir la capacidad de generalización de los modelos.

Mediante un script de preprocesamiento de datos, se separó una fracción de ambos datasets para generar un conjunto de pruebas global y mezclado. Este conjunto de validación contenía tanto tráfico benigno como flujos de ataques SYN Flood e ICMP Flood procedentes de ambas fuentes, sumando un total de **149.396 muestras** de evaluación. Además, se implementó una regla de limpieza para descartar columnas y atributos que no fuesen generalizables o que no se pudieran calcular dinámicamente en tiempo real, definidos en el archivo de texto auxiliar `columns_no_gen_rf.txt`.

Modelo

Se seleccionó el algoritmo **Random Forest** debido a su probada eficacia tabular y su menor propensión al sobreajuste frente a otras alternativas.

Para estudiar el impacto de los datos en la capacidad predictiva, se diseñaron y entrenaron cuatro modelos distintos bajo diferentes estrategias de alimentación de datos:

1. **totalTONIoT**: Entrenado exclusivamente con flujos del dataset TONIoT.
2. **totalPropio**: Entrenado exclusivamente con el dataset del entorno controlado GICAP.
3. **mezclaPropio**: Entrenado con una mezcla de ambos, pero con mayor proporción de flujos del dataset propio.
4. **mezclaTONIoT**: Entrenado con una mezcla de ambos, pero con mayor proporción de flujos de TONIoT.

Durante el preprocesamiento de características en el entrenamiento, se implementaron técnicas de alineación de columnas. Si una característica presente en el entrenamiento faltaba en el flujo de prueba, se rellenaba con ceros, asegurando así que el modelo no fallase en inferencias donde el software de captura no generase un atributo específico.

A continuación, se detallan los resultados obtenidos al crear cada modelo, incluyendo las métricas medias de la validación cruzada y la evaluación interna sobre su propio conjunto de pruebas:

totalTONIoT

■ *Validación cruzada:*

Fold	Accuracy	F1-score	AUC
1	0.975439	0.984334	0.982473
2	0.974824	0.983943	0.981380
3	0.976999	0.985348	0.982971
4	0.976999	0.985352	0.982562
5	0.976321	0.984911	0.982397

Tabla 5.1: Resultados validación cruzada para **totalTONIoT**

- *Evaluación interna:* Al testear el modelo sobre su propio subconjunto de evaluación interna, se alcanza una exactitud (*Accuracy*) global de 0.98 y un F1-score macro de 0.96. El clasificador destaca por una precisión perfecta de 1.00 al etiquetar eventos maliciosos, acompañada de un alto nivel de exhaustividad (*Recall*) de 0.97. La matriz de la figura 5.1 refleja un número extremadamente bajo de falsos positivos (apenas 77 frente a 12.411 verdaderos negativos) y un excelente acierto en la detección, logrando 47.059 verdaderos positivos.

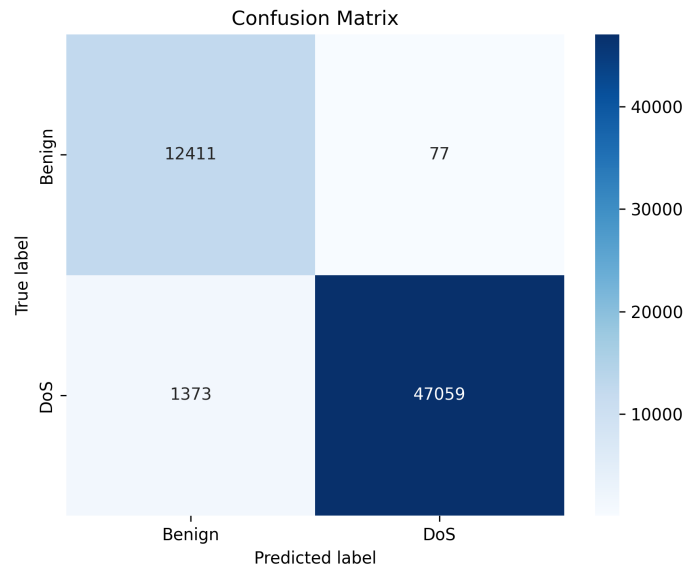


Figura 5.1: Matriz de confusión de la evaluación interna para **totalTONIoT**

totalPropio

- *Validación cruzada:*

Fold	Accuracy	F1-score	AUC
1	0.852113	0.815154	0.890268
2	0.851574	0.814606	0.889867
3	0.852400	0.815451	0.890486
4	0.852510	0.815557	0.890562
5	0.853445	0.816518	0.891265

Tabla 5.2: Resultados validación cruzada para **totalPropio**

- *Evaluación interna:* La inferencia sobre el entorno local registra una exactitud (*Accuracy*) de 0.85 y un F1-score macro de 0.84. Destaca su altísima sensibilidad (*Recall*) de 1.00 para la clase de ataque, logrando detectar la totalidad de las anomalías sin arrojar ningún falso negativo (FN: 0). Sin embargo, esta agresividad predictiva penaliza la precisión, que cae a 0.69 debido a la generación de 50.609 falsos positivos frente a los 111.955 verdaderos positivos.

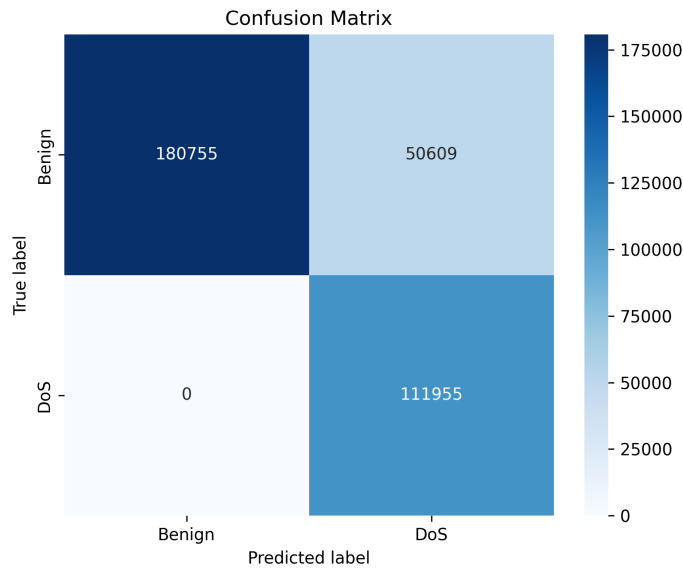


Figura 5.2: Matriz de confusión de la evaluación interna para **totalPropio**

mezclaTONIoT

- *Validación cruzada:*

Fold	Accuracy	F1-score	AUC
1	0.888097	0.890943	0.894937
2	0.889874	0.892623	0.896687
3	0.888535	0.891410	0.895406
4	0.890088	0.892711	0.896818
5	0.888302	0.891173	0.895163

Tabla 5.3: Resultados validación cruzada para **mezclaTONIoT**

- *Evaluación interna:* Al analizar los vectores combinados con mayor dominancia de TONIoT, las métricas globales muestran una exactitud (*Accuracy*) de 0.89 y un índice F1-score macro de 0.89. El modelo mantiene una excelente capacidad de detección con un *Recall* de 0.99 para los ataques, logrando 66.609 verdaderos positivos frente a tan solo 819 falsos negativos. La precisión para la clase maliciosa se establece en 0.81 (debido a 15.497 falsos positivos), logrando un balance muy superior al del modelo totalPropio.

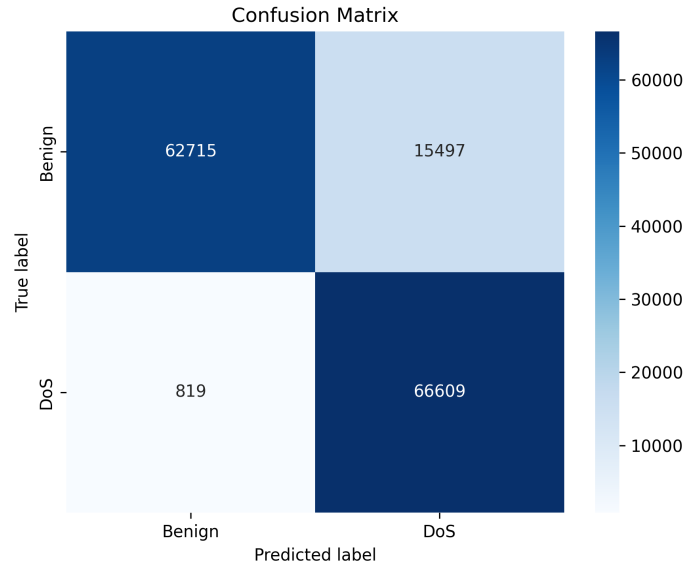


Figura 5.3: Matriz de confusión de la evaluación interna para **mezclaTONIoT**

mezclaPropio

- *Validación cruzada:*

Fold	Accuracy	F1-score	AUC
1	0.860741	0.837106	0.890425
2	0.860537	0.836958	0.890349
3	0.861262	0.837690	0.890951
4	0.860300	0.836728	0.890164
5	0.859904	0.836344	0.889860

Tabla 5.4: Resultados validación cruzada para **mezclaPropio**

- *Evaluación interna:* Los datos mixtos con prevalencia del entorno local resuelven la evaluación interna con una exactitud (*Accuracy*) del 0.86 y un F1-score macro de 0.86. El algoritmo exhibe un desempeño altamente sensible con un *Recall* casi perfecto de 1.00 (identificando 92.588 verdaderos positivos y arrojando solo 325 falsos negativos). Su precisión se sitúa en 0.72, derivada de los 35.630 falsos positivos registrados frente a los 130.057 flujos clasificados correctamente como benignos.

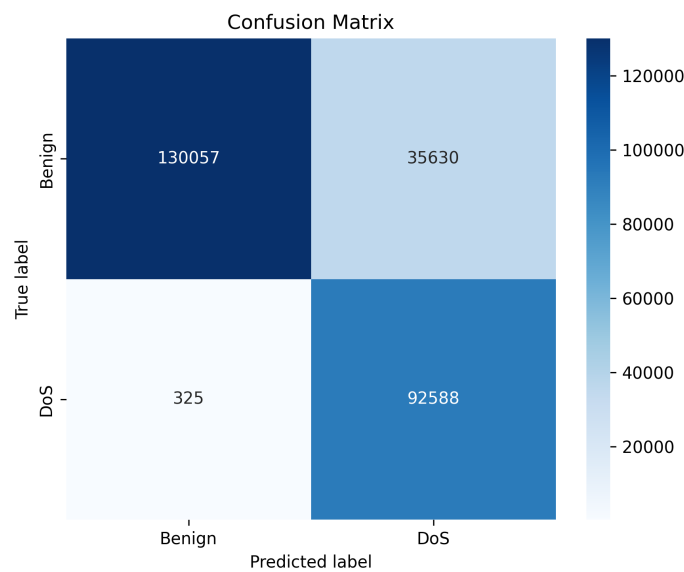


Figura 5.4: Matriz de confusión de la evaluación interna para **mezclaPropio**

Resultados

La evaluación de los cuatro modelos sobre el dataset de prueba mixto, que combina ambas naturalezas de tráfico, arrojó resultados muy dispares que justifican la importancia de diversificar los datos de entrenamiento.

Cuando los modelos se entrenaron con una única fuente de datos, su capacidad de generalizar frente al conjunto de prueba fue deficiente. Por el contrario, los modelos entrenados combinando ambas fuentes de datos lograron un rendimiento óptimo, abarcando de forma exitosa la problemática real.

Los resultados obtenidos para cada uno de los modelos se muestran a continuación:

totalTONIoT

Al emplear únicamente el dataset TONIoT en la fase de entrenamiento, el modelo mejora su capacidad para detectar ataques (Recall de 0.98), pero a costa de clasificar erróneamente una gran cantidad de tráfico legítimo, lo que se refleja en una baja precisión (Precision de 0.72 para ataques).

Clase/Métrica	Precision	Recall	F1-Score	Support
0	0.95	0.48	0.64	63423
1	0.72	0.98	0.83	85973
accuracy	-	-	0.77	149396
macro avg	0.84	0.73	0.73	149396
weighted avg	0.82	0.77	0.75	149396

Tabla 5.5: Resultados evaluación para **totalTONIoT**

totalPropio

Este modelo, entrenado exclusivamente con los flujos del entorno controlado propio, presenta problemas evidentes de generalización al enfrentarse al tráfico mixto. Su desempeño se ve lastrado por una alta tasa de falsos negativos (Recall de 0.40 para la clase de ataque), lo que resulta en una exactitud global deficiente.

Clase/Métrica	Precision	Recall	F1-Score	Support
0	0.55	0.97	0.70	63423
1	0.95	0.40	0.57	85973
accuracy	-	-	0.65	149396
macro avg	0.75	0.69	0.63	149396
weighted avg	0.78	0.65	0.62	149396

Tabla 5.6: Resultados evaluación para **totalPropio**

mezclaTONIoT

El modelo entrenado con una combinación mixta donde predomina el tráfico del dataset TONIoT alcanza el mejor rendimiento global de la comparativa, minimizando tanto los falsos positivos como los negativos y logrando un F1-Score excelente para ambas clases.

Clase/Métrica	Precision	Recall	F1-Score	Support
0	0.98	0.99	0.99	63423
1	0.99	0.99	0.99	85973
accuracy	-	-	0.99	149396
macro avg	0.99	0.99	0.99	149396
weighted avg	0.99	0.99	0.99	149396

Tabla 5.7: Resultados evaluación para **mezclaTONIoT**

mezclaPropio

Al combinar ambas fuentes de datos, manteniendo una predominancia de los flujos propios, el modelo logra generalizar de forma sobresaliente. Las métricas se estabilizan fuertemente, logrando equilibrar la detección de ambas clases con una exactitud global superior al 98 %.

Clase/Métrica	Precision	Recall	F1-Score	Support
0	0.98	0.99	0.98	63423
1	0.99	0.99	0.99	85973
accuracy	-	-	0.99	149396
macro avg	0.99	0.99	0.99	149396
weighted avg	0.99	0.99	0.99	149396

Tabla 5.8: Resultados evaluación para **mezclaPropio**

Estos resultados demuestran que, si solo se tiene en cuenta un entorno de captura, el modelo sobreajusta a la topología específica de esa red. Mezclar datos de distintas fuentes obliga al Random Forest a aprender las características fundamentales del comportamiento de un ataque DoS, en lugar de memorizar artefactos de red propios del entorno de generación.

Detección en tiempo real

El último paso del desarrollo fue la implementación del script de captura e inferencia en el microcontrolador.

Para optimizar los limitados recursos del dispositivo on-edge, el motor de captura de NFStream se configuró para analizar el tráfico de la interfaz de red (WLAN) en ventanas de tiempo dinámicas, introduciendo una regla de

negocio fundamental: únicamente se procesan aquellos flujos cuya dirección IP de destino corresponde a la del propio microcontrolador).

En cada flujo capturado, el script extrae las características estadísticas en el orden exacto en que fueron aprendidas durante el entrenamiento. A continuación, el modelo Random Forest cargado en memoria evalúa las probabilidades del flujo. Si la probabilidad de que el flujo sea malicioso supera el umbral del 0.5, el sistema lo clasifica como **"DoS ATTACK"**.

En caso de detectar una amenaza, el dispositivo no solo registra el ataque localmente junto con la métrica de latencia experimentada, sino que ejecuta una petición *HTTP POST* asíncrona hacia la plataforma web, enviando la IP del atacante y el nivel de certeza del modelo (*score*).

Esta arquitectura permite que un dispositivo aislado on-edge alerte en tiempo real al administrador, cerrando el ciclo entre la detección local y la monitorización centralizada.

6. Trabajos relacionados

Este apartado trata de mencionar algunos de los trabajos y artículos consultados para estudiar el estado del arte y que han influido en el desarrollo del proyecto.

La seguridad en redes de dispositivos IoT y la implementación de sistemas de detección de intrusiones en el extremo de la red (Edge Computing) es un área de investigación muy activa en la actualidad. El incremento exponencial de ataques volumétricos ha impulsado el desarrollo de soluciones que trasladan la carga computacional desde servidores centralizados hacia los propios dispositivos afectados.

La literatura científica existente proporciona una base sólida para el desarrollo de este proyecto, demostrando la eficacia de los algoritmos de aprendizaje automático para analizar flujos de red con un alto grado de precisión.

A continuación, se presentan algunos de los trabajos y publicaciones más relevantes para este proyecto, incluyendo investigaciones previas desarrolladas con el Grupo de Inteligencia Computacional Aplicada (GICAP):

- **Machine Learning for IoT Network Intrusion Detection:** Este artículo presenta un análisis comparativo exhaustivo sobre la eficacia de distintas técnicas de aprendizaje automático aplicadas a sistemas de detección de intrusiones en redes IoT. Los autores evalúan modelos predictivos como Árboles de Decisión (DT), Random Forest (RF), Naive Bayes (NB) y Máquinas de Vectores de Soporte (SVM) sobre el conjunto de datos NSL-KDD, aplicando además técnicas de selección de características como la puntuación de Fisher y la correlación de Pearson. Los resultados experimentales revelan que los modelos basados

en árboles de decisión ofrecen el mejor rendimiento. Esta investigación consolida y respalda la base metodológica de este proyecto, destacando que la familia de algoritmos de árboles de decisión y bosques aleatorios proporciona el equilibrio óptimo entre precisión y carga computacional necesario para dispositivos de recursos limitados. *Fuente:* [7]

- **DDoS Attacks Detection based on Machine Learning Algorithms in IoT Environments:** Este estudio aborda la vulnerabilidad de las infraestructuras IoT y Cloud/Edge frente a ataques distribuidos de Denegación de Servicio (DDoS). Los autores implementan un sistema de detección de intrusiones basado en anomalías, el cual utiliza cálculos de entropía en ventanas temporales de un segundo para evaluar la aleatoriedad del tráfico de red. Al comparar diversos algoritmos de aprendizaje automático y profundo, la investigación demuestra que el algoritmo Random Forest (RF) alcanza un rendimiento sobresaliente. Este trabajo resulta de gran relevancia para el presente proyecto, ya que avala teóricamente la elección del clasificador Random Forest como una solución robusta y de baja latencia para la detección de ataques DoS/DDoS en el extremo de la red. *Fuente:* [11]

- **A Review of Intrusion Detection Systems Using Machine and Deep Learning in Internet of Things: Challenges, Solutions and Future Directions:** En este artículo se aborda el desafío de asegurar las redes de IoT frente a ciberataques mediante la revisión de IDS basados en aprendizaje automático y aprendizaje profundo. El estudio destaca que los métodos tradicionales de detección resultan ineficaces o insuficientes debido a las severas limitaciones de energía, capacidad de cómputo y ancho de banda inherentes a los dispositivos IoT. Tras realizar un análisis exhaustivo de diversas técnicas, la investigación señala que algoritmos como Random Forest resultan especialmente ventajosos para este ecosistema, al ser resistentes al sobreajuste y requerir una menor cantidad de características de entrada sin necesidad de procesos complejos de selección de características. Este trabajo resulta de gran relevancia metodológica para el presente proyecto, ya que fundamenta el problema de la restricción de recursos en el IoT y respalda teóricamente la viabilidad de utilizar clasificadores ligeros en la detección de anomalías directamente en los propios dispositivos. *Fuente:* [5]

Además de estos artículos, para el desarrollo de este proyecto se tuvieron en cuenta dos artículos redactados durante mi periodo de trabajo con el grupo GICAP.

- **IoT Cyberattack Detection in microcontrollers:** En este artículo se abordó el desafío de trasladar modelos de aprendizaje automático a sistemas embebidos. El trabajo se centró en la viabilidad de ejecutar algoritmos de detección de intrusiones utilizando un microcontrolador de arquitectura diferente a la empleada en este proyecto. Esta investigación inicial demostró que los equipos de borde pueden realizar inferencias precisas sobre flujos de red si se optimiza adecuadamente el modelo.. *Fuente:* [2]
- **Real-time identification and evaluation of DoS attacks for embedded systems:** En este trabajo se abordó directamente la problemática de los ataques de Denegación de Servicio (DoS) ejecutando un modelo de inteligencia artificial en un M5Stack LLM630 Compute Kit. La investigación validó la capacidad de este hardware para identificar intrusiones en tiempo real. No obstante, la experimentación se llevó a cabo utilizando exclusivamente tráfico de red simulado dentro de un entorno de laboratorio altamente controlado. Este artículo ha sido aceptado para su publicación en el congreso CISIS 2026.

La revisión y el desarrollo de estos trabajos previos han proporcionado la base teórica, técnica y metodológica necesaria para la concepción de este proyecto.

7. Conclusiones y Líneas de trabajo futuras

7.1. Conclusiones

A lo largo de este proyecto se ha desarrollado, integrado y evaluado con éxito un sistema on-edge completo para la detección de ataques DoS, complementado por una plataforma web de gestión centralizada.

Se puede concluir que el sistema es capaz de distinguir flujos sobre el conjunto de datos utilizado, y se puede considerar su rendimiento como satisfactorio.

Con el proyecto finalizado, se pueden extraer varias conclusiones principales derivadas tanto de los resultados obtenidos como del desarrollo técnico.

Por un lado, se ha logrado trasladar la totalidad de la carga de procesamiento de tráfico y análisis de seguridad al dispositivo microcontrolador (M5Stack LLM 630 Compute Kit) de forma exitosa. El sistema ha demostrado ser capaz de recibir flujos de red en tiempo real, extraer sus características mediante la herramienta NFStream y clasificarlos utilizando el modelo integrado sin intervención de servidores externos. El rendimiento es estable y la latencia obtenida es lo suficientemente baja para realizar una monitorización continua, validando la viabilidad de los equipos de borde para ejecutar tareas críticas de ciberseguridad.

En cuanto a la importancia de las estrategias de entrenamiento, la fase de evaluación, llevada a cabo tanto en pruebas locales como a través de la aplicación, ha revelado que la calidad y naturaleza de los datos de

entrenamiento definen drásticamente la capacidad de generalización del clasificador. Los modelos generados bajo una estrategia de datos mixtos, combinando tráfico de red generado en un entorno controlado con trazas de tráfico real, superan a aquellos entrenados de forma aislada. Esta mezcla de naturalezas permite al modelo abarcar de forma más realista la complejidad del problema, logrando un rendimiento superior y evitando el sobreajuste ante flujos con estructuras variantes.

Además, el desarrollo de la plataforma web ha demostrado ser una solución robusta para solventar la principal limitación de los dispositivos IoT aislados: su actualización y monitorización. La capacidad de desplegar modelos remotamente y evaluar su precisión dota al sistema de un gran valor añadido y escalabilidad operativa.

En conclusión, este proyecto ha supuesto una experiencia especialmente enriquecedora, ya que me ha permitido poner en práctica y profundizar mis conocimientos en técnicas de procesamiento de datos, aprendizaje automático y desarrollo de aplicaciones web. Asimismo, me ha brindado una perspectiva práctica sobre los retos y la complejidad que implica el diseño e implementación de sistemas basados en inteligencia artificial al combinarlos con dispositivos de recursos limitados.

7.2. Líneas de ttrabajo futuras

A pesar de los logros alcanzados y de contar con una prueba de concepto plenamente funcional, el proyecto sienta las bases para diversas ampliaciones y mejoras.

- **Optimización y reducción de la latencia:** Aunque la latencia actual permite un análisis fluido, existe margen de mejora en los tiempos de respuesta. Investigar métodos de preprocesamiento más ligeros que NFStream, optimizar el código base o evaluar el despliegue de los modelos en lenguajes de más bajo nivel (como C/C++) resultaría clave.
- **Transición hacia una visión defensiva:** Actualmente, el dispositivo desempeña un rol de observación y alerta como Sistema de Detección de Intrusiones (IDS). La evolución natural del proyecto consistiría en adoptar una postura defensiva: un Sistema de Prevención de Intrusiones (IPS). Al detectar un flujo malicioso, el microcontrolador debería ser capaz de aplicar reglas de bloqueo inmediatas a nivel de firewall antes

de que el ataque volumétrico logre saturar sus recursos, por ejemplo, bloqueando las direcciones IP atacantes o descartando paquetes. Esto podría ser un punto clave en la reducción de latencia del sistema, ya que al bloquear flujos de ataque, el sistema no desperdiciaría recursos procesando tráfico de IPs maliciosas ya conocidas.

- **Escalabilidad en la plataforma web y ecosistema de detección:**
A nivel de software y gestión, la aplicación web puede ser expandida significativamente. Por un lado, eliminando la restricción al algoritmo Random Forest para permitir la carga y evaluación de arquitecturas de aprendizaje automático y profundo más diversas como Máquinas de Vectores de Soporte o Redes Neuronales. Por otro lado, la plataforma podría aceptar conexiones de múltiples dispositivos simultáneamente, incluyendo aquellos con arquitecturas de hardware distintas al M5Stack, con el objetivo de crear un .ecosistema.º enjambre de detección distribuida, donde la amenaza detectada por un nodo alerte y proteja preventivamente al resto de la red.

Bibliografía

- [1] The TON_IoT Datasets | UNSW Research.
- [2] S. Abejón Pérez, J. A. Rincón, and D. Urda Muñoz. Iot cyberattack detection in microcontrollers. In *International Conference on Intelligent Data Engineering and Automated Learning (IDEAL)*, page 515–525. Springer, 2025. <https://link.springer.com/book/10.1007/978-3-032-10486-1> [Último acceso 3 de junio de 2026].
- [3] Martinez Gonzalez Branly Alberto, Carlos Cambra, Urda Muñoz Daniel, Rincon Arango Jaime Andres, and Alvaro Herrero. Labelled IoT Flow-Based Network Traffic Dataset for Cyberattack Detection, 1 2026.
- [4] Arduino. Portenta h7 | arduino documentation. <https://docs.arduino.cc/hardware/portenta-h7/>, 2026. [Último acceso 18 de febrero de 2026].
- [5] Javed Asharf, Nour Moustafa, Hasnat Khurshid, Essam Debie, Waqas Haider, and Abdul Wahab. A review of intrusion detection systems using machine and deep learning in internet of things: Challenges, solutions and future directions. *Electronics*, 9(7), 2020. <https://www.mdpi.com/2079-9292/9/7/1177> [Último acceso 3 de junio de 2026].
- [6] Luigi Atzori, Antonio Iera, and Giacomo Morabito. The internet of things: A survey. *Computer Networks*, 54(15):2787–2805, 2010. <https://www.sciencedirect.com/science/article/pii/S1389128610001568> [Último acceso 18 de febrero de 2026].
- [7] Marwa Baich, Touria Hamim, Nawal Sael, and Yman Chemlal. Machine learning for iot based networks intrusion detec-

- tion: a comparative study. *Procedia Computer Science*, 215:742–751, 2022. <https://www.sciencedirect.com/science/article/pii/S1877050922021470> [Último acceso 1 de junio de 2026].
- [8] Dave Bergmann. ¿Qué es el machine learning? | IBM — ibm.com. <https://www.ibm.com/es-es/think/topics/machine-learning>. [Último acceso 3 de marzo de 2026].
- [9] Gabriela Bustelo. Qué son los ciberataques activos y pasivos y cómo evitarlos — red seguridad. https://www.redseguridad.com/actualidad/ciberataques-activos-pasivos-que-son-como-evitar_20230921.html, 2023. [Último acceso 2 de marzo de 2026].
- [10] Check Point. Botnet iot - software check point. <https://www.checkpoint.com/es/cyber-hub/network-security/what-is-iot/iot-botnet/>, 2026. [Último acceso 3 de marzo de 2026].
- [11] Mehdi Ebady Manaa, Saba M. Hussain, Suad A. Alasadi, and Hussein A. A. Al-Khamees. Ddos attacks detection based on machine learning algorithms in iot environments. *Inteligencia Artificial*, 27(74):152–165, 2024. [=https://journal.iberamia.org/index.php/intartif/article/view/1394](https://journal.iberamia.org/index.php/intartif/article/view/1394) [Último acceso 1 de junio de 2026].
- [12] Fortinet. ¿qué es un ciberataque y los tipos de ataques en la red? | fortinet. <https://www.fortinet.com/lat/resources/cyberglossary/types-of-cyber-attacks>, 2026. [Último acceso 2 de marzo de 2026].
- [13] Fortinet. ¿qué es un vector de ataque? tipos y cómo evitarlos | fortinet. <https://www.fortinet.com/lat/resources/cyberglossary/attack-vector>, 2026. [Último acceso 2 de marzo de 2026].
- [14] INCIBE. Introducción a los sistemas embebidos | incibe-cert | incibe — incibe.es. <https://www.incibe.es/incibe-cert/blog/introduccion-los-sistemas-embebidos>, 2018. Último acceso 18 de febrero de 2026.
- [15] INCIBE. ¿qué son los ataques dos y ddos? | ciudadanía | incibe. <https://www.incibe.es/ciudadania/blog/que-son-los-ataques-dos-y-ddos>, 2018. [Último acceso 18 de febrero de 2026].
- [16] INCIBE. Las 7 fases de un ciberataque. ¿las conoces? <https://www.incibe.es/empresas/blog/las-7-fases-ciberataque-las-conoces>, 2020. [Último acceso 25 de febrero de 2026].

- [17] Inforges. ¿qué es un exploit en ciberseguridad y para qué sirve? - inforges. <https://inforges.es/blog/que-es-un-exploit-en-ciberseguridad/>, 2026. [Último acceso 2 de marzo de 2026].
- [18] Kaspersky. ¿qué es un ataque ddos? definición, significado, tipos — kaspersky. <https://www.kaspersky.es/resource-center/threats/ddos-attacks>, 2026. [Último acceso 18 de febrero de 2026].
- [19] Eda Kavlakoglu. ¿Qué es random forest? | IBM — ibm.com. <https://www.ibm.com/mx-es/think/topics/random-forest>. [Último acceso 3 de marzo de 2026].
- [20] Constantinos Kolias, Georgios Kambourakis, Angelos Stavrou, and Jeffrey M. Voas. Ddos in the iot: Mirai and other botnets. *Computer*, 50(7):80–84, 2017. <https://ieeexplore.ieee.org/document/7971869> [Último acceso 18 de febrero de 2026].
- [21] M5Stack. Core2 - m5-docs - m5stack. <https://docs.m5stack.com/en/core/core2>, 2026. [Último acceso 18 de febrero de 2026].
- [22] M5Stack. Llm630 compute kit config — m5-docs - m5stack. https://docs.m5stack.com/en/guide/llm/llm630_compute_kit/config, 2026. [Último acceso 18 de febrero de 2026].
- [23] Redacción. ¿qué es el malware? tipos y maneras de evitar ataques de este tipo — red seguridad. https://www.redseguridad.com/actualidad/cibercrimen/que-es-el-malware-tipos-y-maneras-de-evitar-ataques-de-este-tipo_20241130.html, 2024. [Último acceso 3 de marzo de 2026].